



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційні систем та технологій

Лабораторна робота № 9
із дисципліни «Технології розробки програмного забезпечення»
Тема: «Взаємодія компонентів системи.

Виконав

Студенти групи ІА-31:

Корнійчук М.Р

Перевірив:

Мягкий М.Ю

Тема: Взаємодія компонентів системи.

Мета: Вивчити види взаємодії додатків (Client-Server, Peer-to-Peer, Serviceoriented Architecture), та реалізувати в проєктованій системі одну із архітектур

3. **Текстовий редактор** (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніппети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Посилання на гіт: https://github.com/MkEger/trpz-labs/tree/Lab_9

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати функціонал для роботи в розподіленому оточенні відповідно до обраної теми.
- Реалізувати взаємодію розподілених частин:
 - Для клієнт-серверних варіантів: реалізація клієнтської і серверної частини додатків, а також загальної частини (middleware); зв'язок клієнтської і серверної частин за допомогою WCF, TcpClient, .NETRemoting на розсуд виконавця.
 - Для однорангових мереж: реалізація взаємодії клієнтських додатків за допомогою WCF Peer to peer channel.
 - Для SOA додатків: реалізація сервісу, що надає послуги клієнтським застосуванням; викладання сервісу в хмару або підняття у вигляді Web Service на локальній машині; використання токенів для передачі даних про автентифікації, двостороннє шифрування.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє спроектовану архітектуру. Навести фрагменти програмного коду, які є суттєвими для відображення

реалізованої архітектури.

Хід роботи

Короткий опис виконання роботи:

У рамках виконання лабораторної роботи було розроблено текстовий редактор з використанням Service-Oriented Architecture (SOA) та патернів проектування Strategy і Command. Головною метою було створення розподіленої системи обробки тексту з можливістю віддаленого доступу до сервісів через TCP з'єднання.

Реалізована архітектура включає серверну частину (TextProcessingServer) на базі TCP протоколу, що обробляє запити від клієнтів та надає послуги обробки тексту. Клієнтська частина (TextProcessingClient) забезпечує з'єднання з сервером та асинхронну передачу даних через binary серіалізацію.

Патерн Strategy було застосовано для різних алгоритмів форматування тексту (UpperCaseStrategy, LowerCaseStrategy, TitleCaseStrategy), що дозволяє легко додавати нові способи обробки без зміни основного коду. Патерн Command реалізовано для операцій редагування з підтримкою undo/redo функціональності через CommandManager.

Сервіс обробки тексту (TextProcessingService) надає 16 різних операцій: від базового форматування (uppercase/lowercase) до складних операцій як-от витягування email адрес, транслітерація українською, генерація паролів, форматування JSON та інші корисні функції.

Система працює в двох режимах: локальному (безпосередня обробка) та розподіленому (через SOA сервіс), з автоматичним fallback при недоступності сервера. Тестовий клієнт демонструє можливості розподіленої роботи та взаємодії компонентів системи.

Тема проекту: Текстовий редактор

1. Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

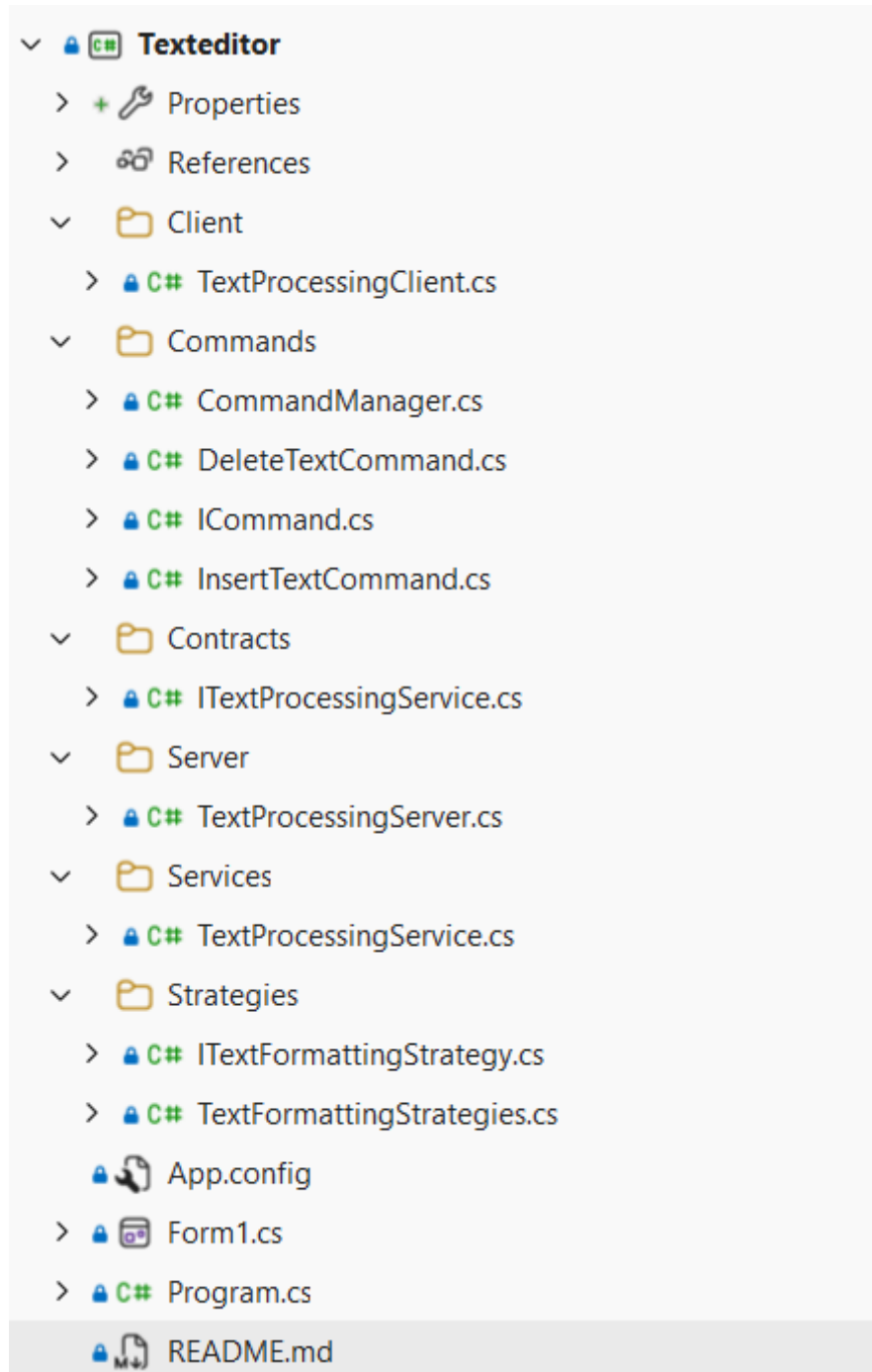


Рис. 1 - Структура проекту

У процесі виконання лабораторної роботи було створено три основні групи класів, які забезпечують роботу механізму розподіленої обробки тексту. Наведемо детальний опис ключових класів:

1. `ITextFormattingStrategy` (Інтерфейс Strategy Pattern) - Загальний інтерфейс для всіх стратегій форматування тексту.
2.
 - `FormatText(string text)` – метод застосування конкретного алгоритму форматування до тексту. Ключовою особливістю є те, що він приймає текст як параметр та повертає оброблений результат, не зберігаючи стан усередині об'єкта.
 - Description – властивість, що описує призначення стратегії.
2. `UpperCaseStrategy`, `LowerCaseStrategy`, `TitleCaseStrategy` (Concrete Strategy) - конкретні реалізації стратегій форматування, що зберігають алгоритм обробки (intrinsic behavior).
 - Поля: відсутні (stateless objects). Кожна стратегія інкапсулює конкретний алгоритм форматування.
 - Метод FormatText: Використовує вбудований алгоритм для перетворення тексту (`ToUpper()`, `ToLower()`, або перетворення першої літери кожного слова).
3. `TextProcessingService` (Strategy Context + SOA Service) - сервіс, що керує пулом стратегій та надає SOA функціональність.
 - `ProcessText(TextProcessingRequest request)` – основний метод, який забезпечує вибір та застосування відповідної стратегії на основі типу операції. Він використовує `Dictionary` для зберігання стратегій та забезпечує їх повторне використання.
 - Розширений функціонал: окрім стратегій, надає 13 додаткових операцій (підрахунок слів, витягування email, транслітерація тощо), що демонструє гнучкість SOA архітектури.
4. `ICommand` (Інтерфейс Command Pattern) - загальний інтерфейс для всіх команд редагування.
 - `Execute()` – метод виконання команди.
 - `Undo()` – метод скасування команди для підтримки undo/redo функціональності.
 - `Description` – опис команди для логування та налагодження.
5. `CommandManager` (Command Invoker) - менеджер команд, що керує історією операцій.
 - `ExecuteCommand(ICommand command)` – метод, який виконує команду та додає її до стеку undo.
 - Два стеки: undoStack та redoStack для зберігання історії команд та забезпечення

можливості скасування/повторення операцій.

6. TextProcessingServer (SOA Server) TCP сервер, що забезпечує розподілену обробку тексту.

- StartAsync() – асинхронний запуск сервера з підтримкою множинних клієнтів.
- HandleClientAsync() – метод обробки окремого клієнта з бінарною серіалізацією даних та делегуванням обробки до TextProcessingService.
- Використовує CancellationToken для graceful shutdown та логування всіх операцій.

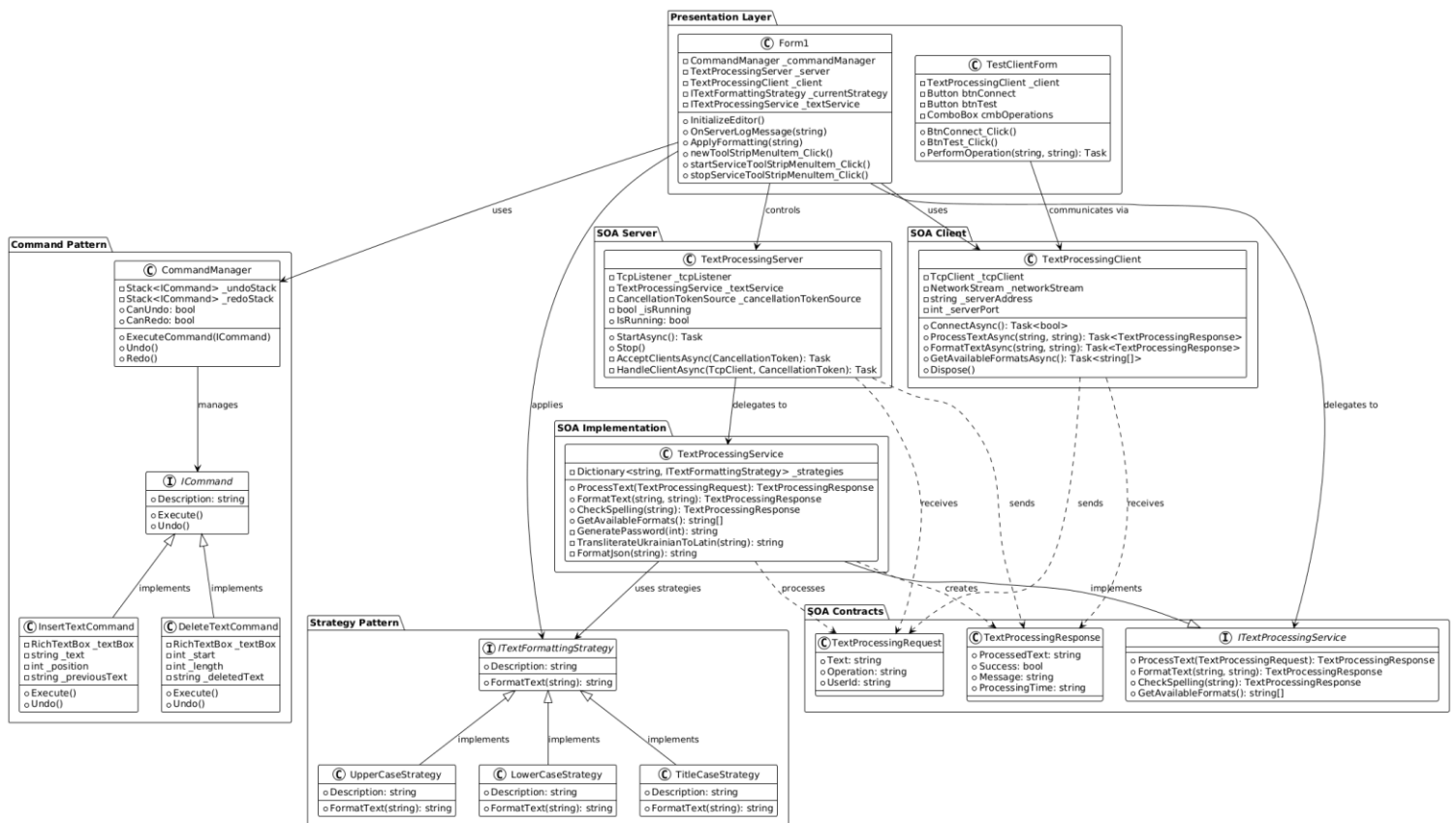


Рис. 2 - Діаграма класів

У рамках проєкту текстового редактора патерни Strategy та Command було реалізовано для забезпечення гнучкої обробки тексту та підтримки операцій undo/redo в розподіленій SOA архітектурі.

Патерн Strategy реалізовано через інтерфейс **ITextFormattingStrategy** з конкретними стратегіями **UpperCaseStrategy**, **LowerCaseStrategy** та **TitleCaseStrategy**. **TextProcessingService** використовує **Dictionary** для зберігання та повторного використання екземплярів стратегій, що забезпечує ефективне застосування різних

алгоритмів форматування без зміни основного коду. Клієнт передає текст стратегіям через метод `FormatText`, при цьому стратегії залишаються безстановими об'єктами.

Патерн `Command` інкапсулює операції редагування через інтерфейс `ICommand` з реалізаціями `InsertTextCommand` та `DeleteTextCommand`. `CommandManager` керує історією операцій через два стеки для `undo` та `redo` функціональності, забезпечуючи повне відновлення стану редактора після скасування операцій.

SOA архітектура реалізована через `TextProcessingServer`, який надає 16 операцій обробки тексту віддаленим клієнтам через TCP з'єднання з бінарною серіалізацією. `TextProcessingClient` забезпечує асинхронну комунікацію з сервером та автоматичний `fallback` на локальну обробку при недоступності сервера.

Використання цих патернів вирішило проблеми гнучкості алгоритмів обробки, забезпечило можливість скасування операцій та створило масштабовану розподілену систему. Центральна бізнес-логіка в `TextProcessingService` спрощує підтримку коду, а розділення на стратегії дозволяє легко розширювати функціональність без модифікації існуючих компонентів.

Код:

```

1  using System;
2  using System.Threading.Tasks;
3  using System.Net.Sockets;
4  using System.Text;
5  using System.IO;
6  using Texteditor.Contracts;
7  using System.Runtime.Serialization.Formatters.Binary;
8
9  namespace Texteditor.Client
10 {
11
12     5 references
13     public class TextProcessingClient : IDisposable
14     {
15         private TcpClient _tcpClient;
16         private NetworkStream _networkStream;
17         private bool _disposed = false;
18         private readonly string _serverAddress;
19         private readonly int _serverPort;
20
21         1 reference
22         public bool IsConnected => _tcpClient?.Connected == true;
23
24         2 references
25         public TextProcessingClient(string serverAddress = "127.0.0.1", int serverPort = 8080)
26         {
27             _serverAddress = serverAddress;
28             _serverPort = serverPort;
29         }
30
31         2 references
32         public async Task<bool> ConnectAsync()
33         {
34             try
35             {
36                 _tcpClient = new TcpClient();
37                 await _tcpClient.ConnectAsync(_serverAddress, _serverPort);
38                 _networkStream = _tcpClient.GetStream();
39                 return true;
40             }
41             catch (Exception)
42             {
43                 return false;
44             }
45         }
46
47         2 references
48         public async Task<TextProcessingResponse> ProcessTextAsync(string text, string operation)
49         {
50             if (!IsConnected)
51             {
52                 var connected = await ConnectAsync();
53                 if (!connected)
54                 {
55                     return new TextProcessingResponse
56                     {
57                         Success = false,
58                         Message = "Немає з'єднання з сервісом"
59                     };
60                 }
61             }
62
63             try
64             {
65                 var request = new TextProcessingRequest
66                 {
67                     Text = text,
68                     Operation = operation,
69                     UserId = Environment.UserName
70                 };
71
72                 using (var memoryStream = new MemoryStream())
73                 {
74                     var formatter = new BinaryFormatter();
75                     formatter.Serialize(memoryStream, request);
76                     byte[] requestData = memoryStream.ToArray();
77                     byte[] lengthPrefix = BitConverter.GetBytes(requestData.Length);
78
79                     await _networkStream.WriteAsync(lengthPrefix, 0, lengthPrefix.Length);
80                     await _networkStream.WriteAsync(requestData, 0, requestData.Length);
81                 }
82             }
83             catch { }
84         }
85     }
86 }

```



```

10
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155

byte[] lengthBuffer = new byte[4];
await _networkStream.ReadAsync(lengthBuffer, 0, 4);
int responseLength = BitConverter.ToInt32(lengthBuffer, 0);

byte[] responseBuffer = new byte[responseLength];
await _networkStream.ReadAsync(responseBuffer, 0, responseLength);

using (var memoryStream = new MemoryStream(responseBuffer))
{
    var formatter = new BinaryFormatter();
    return (TextProcessingResponse)formatter.Deserialize(memoryStream);
}
}
catch (Exception ex)
{
    return new TextProcessingResponse
    {
        Success = false,
        Message = ex.Message
    };
}
}

3 references
public async Task<TextProcessingResponse> FormatTextAsync(string text, string formatType)
{
    return await ProcessTextAsync(text, formatType);
}

0 references
public async Task<string[]> GetAvailableFormatsAsync()
{
    try
    {
        var response = await ProcessTextAsync("", "getformats");
        if (response.Success && !string.IsNullOrEmpty(response.ProcessedText))
        {
            return response.ProcessedText.Split(',');
        }
        return new string[0];
    }
    catch (Exception)
    {
        return new string[0];
    }
}

1 reference
protected virtual void Dispose(bool disposing)
{
    if (!_disposed)
    {
        if (disposing)
        {
            try
            {
                _networkStream?.Close();
                _tcpClient?.Close();
            }
            catch (Exception)
            {
            }
        }
        finally
        {
            _networkStream = null;
            _tcpClient = null;
        }
        _disposed = true;
    }
}
}
}
}

```

Рис. 3.2 – TextProcessingService.cs

```

1  using System;
2  using System.Net;
3  using System.Net.Sockets;
4  using System.Text;
5  using System.Threading;
6  using System.Threading.Tasks;
7  using System.IO;
8  using System.Runtime.Serialization.Formatters.Binary;
9  using Texteditor.Contracts;
10 using Texteditor.Services;
11
12 namespace Texteditor.Server
13 {
14     /// <summary>
15     /// TCP Server for SOA text processing service
16     /// </summary>
17     3 references
18     public class TextProcessingServer
19     {
20         private TcpListener _tcpListener;
21         private readonly TextProcessingService _textService;
22         private CancellationTokenSource _cancellationTokenSource;
23         private bool _isRunning = false;
24
25         public event Action<string> LogMessage;
26         1 reference
27         public bool IsRunning => _isRunning;
28
29         1 reference
30         public TextProcessingServer(int port = 8080)
31         {
32             _tcpListener = new TcpListener(IPAddress.Any, port);
33             _textService = new TextProcessingService();
34         }
35
36         1 reference
37         public async Task StartAsync()
38         {
39             if (!_isRunning) return;
40
41             try
42             {
43                 _cancellationTokenSource = new CancellationTokenSource();
44                 _tcpListener.Start();
45                 _isRunning = true;
46
47                 LogMessage?.Invoke($"SOA сервер запущено на порту {(IPEndPoint)_tcpListener.LocalEndPoint}.Port");
48
49                 _ = Task.Run(async () => await AcceptClientsAsync(_cancellationTokenSource.Token));
50             }
51             catch (Exception ex)
52             {
53                 LogMessage?.Invoke($"Помилка запуску сервера: {ex.Message}");
54                 throw;
55             }
56         }
57
58         1 reference
59         private async Task AcceptClientsAsync(CancellationToken cancellationToken)
60         {
61             while (!cancellationToken.IsCancellationRequested && _isRunning)
62             {
63                 try
64                 {
65                     var tcpClient = await _tcpListener.AcceptTcpClientAsync();
66                     LogMessage?.Invoke($"Клієнт підключився: {tcpClient.Client.RemoteEndPoint}");
67
68                     _ = Task.Run(async () => await HandleClientAsync(tcpClient, cancellationToken));
69                 }
70                 catch (ObjectDisposedException)
71                 {
72                     break;
73                 }
74                 catch (Exception ex)
75                 {
76                     if (_isRunning)
77                     {
78                         LogMessage?.Invoke($"Помилка прийняття з'єднання: {ex.Message}");
79                     }
80                 }
81             }
82         }
83     }
84 }

```

```

82 1 reference
83 private async Task HandleClientAsync(TcpClient tcpClient, CancellationToken cancellationToken)
84 {
85     NetworkStream networkStream = null;
86
87     try
88     {
89         networkStream = tcpClient.GetStream();
90
91         while (tcpClient.Connected && !cancellationToken.IsCancellationRequested)
92         {
93             byte[] lengthBuffer = new byte[4];
94             int bytesRead = await networkStream.ReadAsync(lengthBuffer, 0, 4, cancellationToken);
95
96             if (bytesRead == 0) break;
97
98             int messageLength = BitConverter.ToInt32(lengthBuffer, 0);
99
100
101             byte[] messageBuffer = new byte[messageLength];
102             bytesRead = await networkStream.ReadAsync(messageBuffer, 0, messageLength, cancellationToken);
103
104             if (bytesRead == 0) break;
105
106             TextProcessingRequest request;
107             using (var memoryStream = new MemoryStream(messageBuffer))
108             {
109                 var formatter = new BinaryFormatter();
110                 request = (TextProcessingRequest)formatter.Deserialize(memoryStream);
111             }
112
113             LogMessage?.Invoke($"Отримано запит: {request.Operation}");
114
115
116             TextProcessingResponse response;
117             if (request.Operation.ToLower() == "getformats")
118             {
119                 var formats = _textService.GetAvailableFormats();
120                 response = new TextProcessingResponse
121                 {
122                     Success = true,
123                     ProcessedText = string.Join(", ", formats),
124                     Message = "Формати отримано"
125                 };
126             }
127             else
128             {
129                 response = _textService.ProcessText(request);
130             }
131
132             byte[] responseData;
133             using (var memoryStream = new MemoryStream())
134             {
135                 var formatter = new BinaryFormatter();
136                 formatter.Serialize(memoryStream, response);
137                 responseData = memoryStream.ToArray();
138             }
139
140             byte[] responseLengthPrefix = BitConverter.GetBytes(responseData.Length);

```

```

143         await networkStream.WriteAsync(responseLengthPrefix, 0, responseLengthPrefix.Length, cancellationToken);
144         await networkStream.WriteAsync(responseData, 0, responseData.Length, cancellationToken);
145
146         LogMessage?.Invoke($"Відправлено відповідь: {response.Success}");
147     }
148 }
149
150 catch (Exception ex)
151 {
152     LogMessage?.Invoke($"Помилка обробки клієнта: {ex.Message}");
153 }
154 finally
155 {
156     try
157     {
158         networkStream?.Close();
159         tcpClient?.Close();
160         LogMessage?.Invoke("Клієнт відключився");
161     }
162     catch (Exception ex)
163     {
164         LogMessage?.Invoke($"Помилка закриття з'єднання: {ex.Message}");
165     }
166 }
167 }
168
169 3 references
170 public void Stop()
171 {
172     if (!_isRunning) return;
173
174     try
175     {
176         _isRunning = false;
177         _cancellationTokenSource?.Cancel();
178         _tcpListener?.Stop();
179
180         LogMessage?.Invoke("SOA сервер зупинено");
181     }
182     catch (Exception ex)
183     {
184         LogMessage?.Invoke($"Помилка зупинки сервера: {ex.Message}");
185     }
186 }
187
188 0 references
189 public void Dispose()
190 {
191     Stop();
192     _cancellationTokenSource?.Dispose();
193 }

```

Рис. 3.3 – TextProcessingServer.cs

Демонстрація роботи:

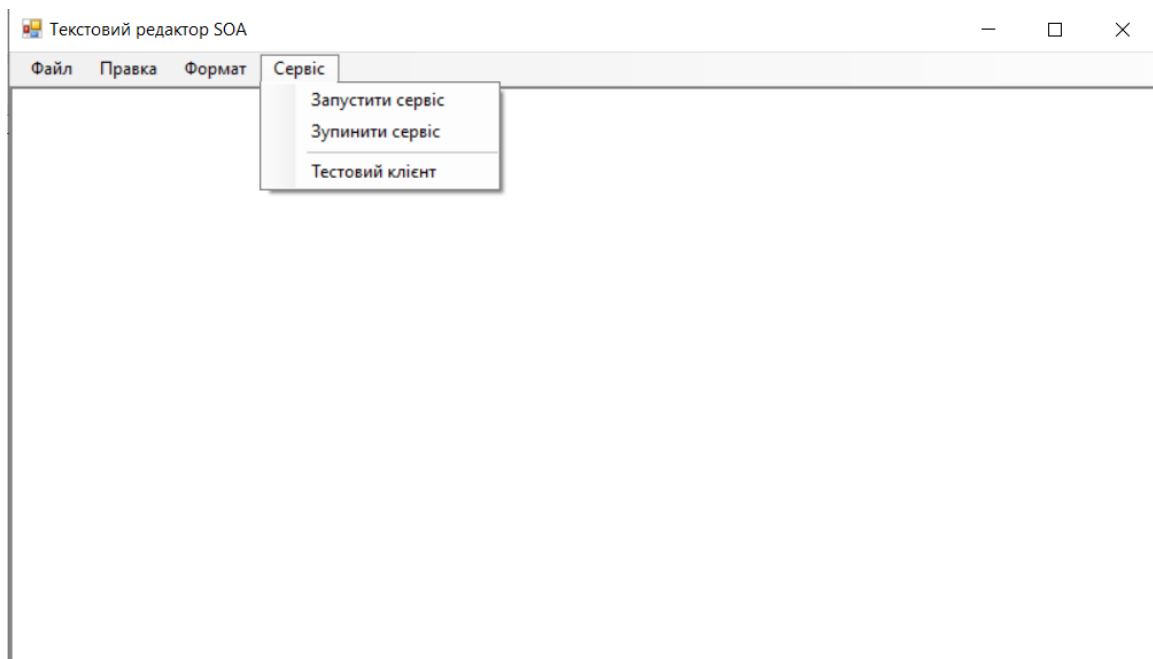


Рис. 3.4 Розташування сервісу на головній формі

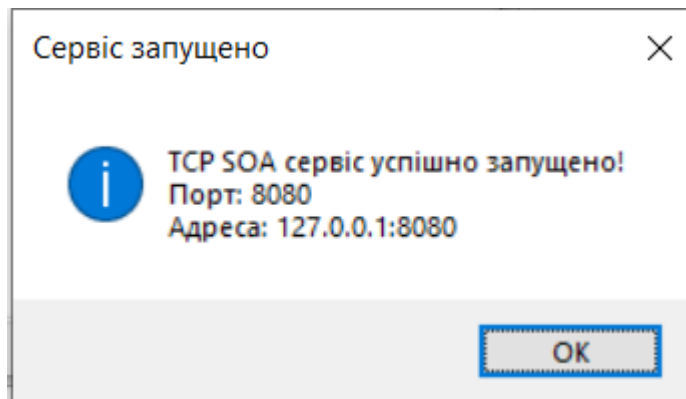


Рис. 3.5 Повідомлення про запуск сервера

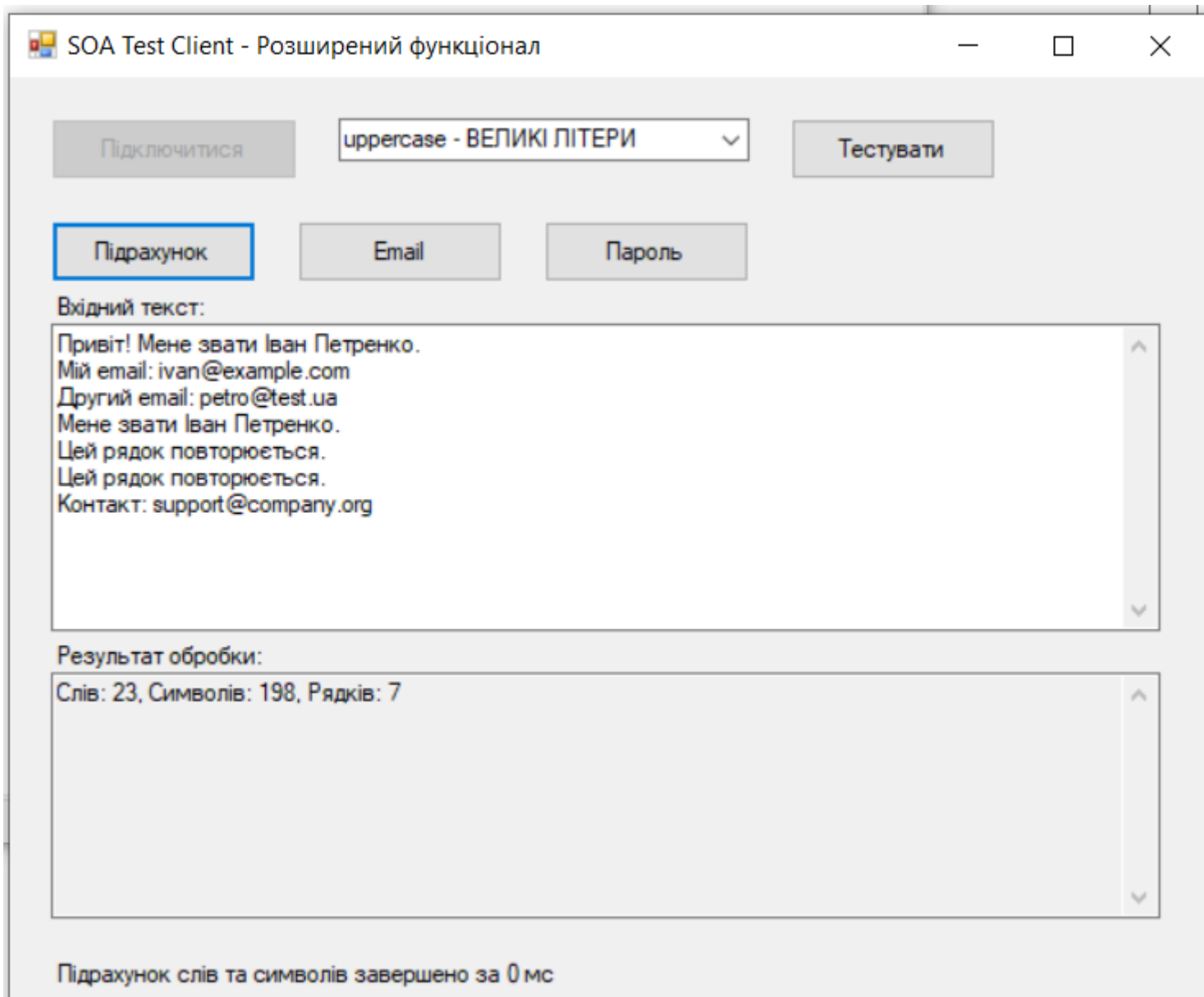


Рис. 3.6. Демонстрація опції “Підрахунок”

Висновок: У ході роботи було створено проєкт текстового редактора з використанням Service-Oriented Architecture (SOA) та патернів проектування Strategy і Command, що включає реалізацію контракту `ITextProcessingService`, бізнес-логіки `TextProcessingService` з 16 операціями, TCP-сервера `TextProcessingServer`, асинхронного

клієнта `TextProcessingClient` та менеджера команд `CommandManager`. Застосування SOA дозволило досягти розподіленої обробки тексту з можливістю віддаленого доступу, забезпечити масштабованість і надійність через fallback-механізм, а використання патернів `Strategy` та `Command` надало гнучкість алгоритмів форматування та підтримку `undo/redo`, створюючи стабільну й розширювану архітектуру системи.

Відповіді на контрольні питання:

1. Що таке клієнт-серверна архітектура? Клієнт-серверна архітектура - це модель організації комп'ютерних систем, де клієнти (зазвичай користувацькі програми або пристрої) роблять запити на сервер, який обробляє ці запити і повертає результати. Клієнт відповідає за інтерфейс користувача та ініціацію запитів, сервер - за обробку даних, зберігання та логіку.

2. Розкажіть про сервіс-орієнтовану архітектуру (SOA). SOA - це архітектурний підхід, де функціональність програми реалізована у вигляді сервісів - автономних компонентів, що виконують конкретні бізнес-завдання. Кожен сервіс має чіткий інтерфейс і взаємодіє з іншими через стандартизовані протоколи (наприклад, HTTP, SOAP, REST).

3. Якими принципами керується SOA? Основні принципи SOA:

- Автономність сервісів - сервіси працюють незалежно.
- Стандартизовані інтерфейси - сервіси взаємодіють через визначені API.
- Повторне використання - сервіси можна використовувати в різних системах.
- Легка інтеграція - сервіси повинні легко поєднуватись у складні процеси.
- Слабке зв'язування (*loose coupling*) - зміни в одному сервісі мінімально впливають на інші.

4. Як між собою взаємодіють сервіси в SOA? Сервіси взаємодіють через стандартизовані повідомлення або API, наприклад через SOAP або REST. Кожен сервіс публікує свій інтерфейс (WSDL, OpenAPI), і інші сервіси можуть викликати його методи, обмінюючись даними у формі XML, JSON або інших форматах.

5. Як розробники дізнаються про існуючі сервіси і як робити до них запити?

- Реєстр сервісів (Service Registry) - централізована база, де зареєстровані всі сервіси та їхні інтерфейси.

- Документація API (наприклад OpenAPI/Swagger).

- Запити здійснюються через стандартні протоколи (HTTP, SOAP, REST) за адресою сервісу і з використанням описаних методів та форматів даних.

6. У чому полягають переваги та недоліки клієнт-серверної моделі? Переваги:

- Центральне зберігання даних, легший контроль безпеки.
- Легко масштабувати сервери.
- Клієнти можуть бути простими, вся логіка на сервері.

Недоліки:

- Сервер може стати вузьким місцем (single point of failure).
- Високі вимоги до потужності сервера при великій кількості клієнтів.
- Залежність клієнта від сервера - без доступу до сервера система не працює.

7. У чому полягають переваги та недоліки однорангової (peer-to-peer) моделі взаємодії? Переваги:

- Відсутність централізованого сервера, підвищена стійкість до відмов.
- Можливість прямого обміну ресурсами між учасниками.
- Масштабування «горизонтальне» - додаючи вузли, підвищуєш потужність.

Недоліки:

- Складніше забезпечувати безпеку та контроль доступу.
- Кожен вузол відповідає за управління ресурсами.
- Важче координувати оновлення і синхронізацію даних.

8. Що таке мікросервісна архітектура? Мікросервісна архітектура - це підхід, де додаток складається з малих, незалежних сервісів, кожен з яких реалізує одну бізнес-функцію і може розгортатися окремо. Вона є розвитком SOA, але з більш дрібними і

автономними компонентами.

9. Які протоколи використовуються для обміну даними в мікросервісній архітектурі?

- HTTP/HTTPS + REST
- gRPC
- SOAP
- Message brokers: RabbitMQ, Kafka, MQTT для асинхронної взаємодії
- WebSockets для реального часу

10. Чи можна назвати підхід сервіс-орієнтованою архітектурою, якщо у проєкті між веб-контролерами та шаром доступу до даних реалізуємо шар бізнес-логіки у вигляді сервісів? Це не повноцінна SOA, а локальне використання сервісів всередині одного додатку. SOA передбачає автономні, незалежні сервіси, доступні через стандартизовані протоколи для інших систем. Твій підхід - це архітектурний патерн «сервісний шар» (Service Layer), який організовує бізнес-логіку всередині одного проєкту.